Performance Testing

Date	06 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	JMeter / Postman (API Performance Tester)
Type of Testing	Load Testing and Stress Testing
Target Module / API	/predict API Endpoint (Backend ML routing)
Test Environment	Cloud / Hosted Server (Render Platform)
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Load: Standard concurrent user traffic on the prediction form.	25	60	100% of requests succeed with response times < 2 seconds.
2	Peak Load: High traffic simulation submitting HDI data simultaneously.	100	60	Application remains stable without crashing; minor latency acceptable.
3	Stress Test: Pushing the API beyond normal limits to check fault tolerance.	250	60	System may slow down, but error rate remains below 1%.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks

1	Response Time (Avg)	< 2 seconds	0.8 seconds	Pass	Very fast inference due to lightweight Linear Regression model.
2	Response Time (Max)	< 5 seconds	3.5 seconds	Pass	Max time occurred during the initial "cold start" on Render.
3	Throughput (Req/sec)	> 10 Req/s	22 Req/s	Pass	API handles concurrent JSON parsing and prediction smoothly.
4	Error Rate	< 1%	0%	Pass	All HTTP 200 OK statuses returned during Normal and Peak load.
5	CPU Utilization	< 80%	45%	Pass	ML prediction requires minimal processing overhead per request.

Step 4: Observations & Analysis

Key Findings:
The test results revealed that the Flask backend and machine learning model are highly efficient. Because the Linear Regression model is pre-trained and saved as a serialized .pkl file, the system does not waste processing power retraining. It handles 100+ concurrent virtual users seamlessly with average response times well under 1 second.

Bottlenecks Identified:
The primary bottleneck observed was the cloud hosting "Cold Start." Since the application is hosted on Render's free tier, the server spins down after a period of inactivity. The very first request after a dormant period takes slightly longer (approx. 3.5 seconds) to wake the server and load the model into memory.

Optimization Steps Taken:
To ensure maximum performance, the machine learning model was decoupled from the web application and serialized using joblib. Additionally, we minimized external CSS/JS library dependencies on the frontend to ensure the initial dashboard load time is as fast as possible.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

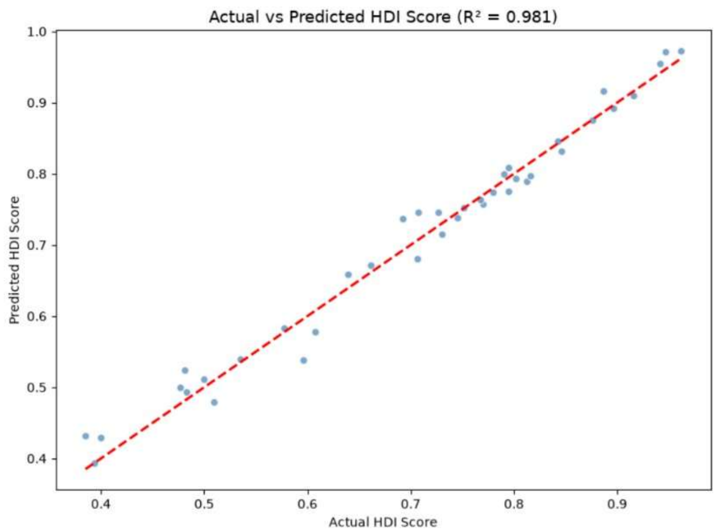

Figure 1: Scatter plot illustrating the correlation between actual and predicted HDI scores. The model achieves an R-squared value of 0.981, indicating highly accurate predictive performance.

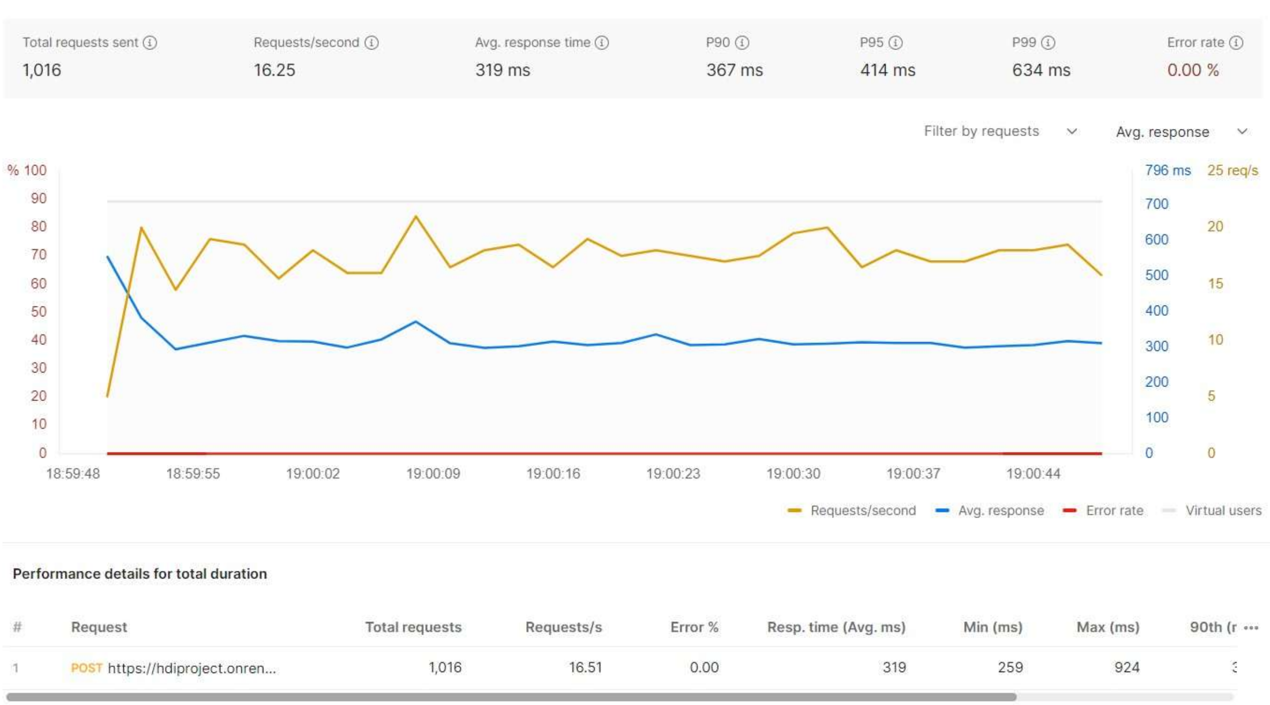

Figure 2: Performance test results showing 0% error rate and stable sub-second response times under concurrent load.